

中山大学计算机学院

编译器构造实验 Lab2 实验报告

ExprEval: 基于 OPP 的表达式求值器

本报告对应 `parser.Calculator#calculate(String)` 的最终提交版本。内部实现已重做为 词法扫描 + 算符优先移进归约分析 + 归约期语义检查，同时补齐 `mytest.xml`、`readme.txt`、`design.pdf` 和 Javadoc。

课程	编译器构造实验 (DCS292)
实验名称	Lab2 表达式求值器 ExprEval
姓名	梁力航
学号	23336128
日期	2026 年 4 月 22 日
运行环境	OpenJDK 11.0.24 / macOS / XeLaTeX

解析方法

OPP

源码文件

7 个

standard.xml

16 / 16

mytest.xml

15 / 15

提交物

**PDF + TXT
+ XML**

目录

1	实验目标与本次重做范围	2
2	二义性分析	2
3	词法分析设计与实现	3
3.1	单词类别和边界规则	3
3.2	数值扫描自动机	3
3.3	词法实现细节	3
4	算符优先语法分析与语义处理	4
4.1	模块划分	4
4.2	优先级与结合性	5
4.3	压缩后的 OPP 关系表	5
4.4	移进/归约流程	6
4.5	语义规则与异常落点	7
5	自定义测试用例说明	8
6	程序运行截图	8
6.1	GUI 运行界面	8
6.2	测试结果截图	9
7	文档与源程序组织说明	10
8	实验心得	10

1 实验目标与本次重做范围

本次提交不再沿用之前的递归下降主体，而是按实验 PDF 和评分表重做为 OPP 版本，范围如下：

- 对外接口保持不变，软装置仍只调用 `parser.Calculator#calculate(String)`；
- 词法分析单独拆分为 `Scanner`，严格执行 PDF 中“只有空格是合法分隔符”的约束；
- 语法分析改为栈式移进/归约，不再依赖 `parseConditional()` 这一类递归下降链；
- 语义检查绑定在归约动作中完成，保证异常落在最具体的异常类上；
- 提交目录补齐 `mytest.xml`、`readme.txt`、`design.pdf`、`Javadoc`。

这次实现的重点不是“算出来”，而是“按 PDF 要求算出来”。因此实现里最先固定的是词法边界、运算优先级、三元配对规则和异常落点，而不是代码行数。

2 二义性分析

实验 PDF 给出的表达式语言本身可以定义清楚，但直接照 BNF 落地时，至少有三类位置会在实现层面产生歧义：

1. 一元负号和二元减号共用字符 `-`。例如 `5--2.5` 和 `2-3*-4` 都合法，只有结合上下文才能知道当前 `-` 是前缀运算符还是中缀运算符。
2. 幂运算 `^` 是右结合。如果按普通左结合二元运算处理，`2 ^ 3 ^ 2` 会被错误地解释为 `(2 ^ 3) ^ 2`。
3. 三元运算 `?:` 的配对不是局部的。例如 `a ? b : c ? d : e` 必须按右结合解释为 `a ? b : (c ? d : e)`，而 `6 ? 7 : 7 : 9` 又必须在第二个 `:` 处及时报错。

因此，这门语言虽然不是“无法描述”，但如果直接用原始 BNF 去写朴素自顶向下实现，会把优先级、结合性和配对规则散落在很多分支里。这里改用 OPP 的原因很直接：

- 运算优先级统一交给一张表；
- 归约点统一由“栈顶运算符 vs 当前输入符号”决定；
- 一元负号、函数参数、三元配对都可以在移进/归约框架里处理，不需要额外再嵌一套解析主流程。

3 词法分析设计与实现

3.1 单词类别和边界规则

类别	例子	本次实现中的处理规则
空白分隔符		只接受空格; \t、\n、\r 一律抛 <code>IllegalSymbolException</code>
数值常量	12、3.14、2.25E+2	只允许无符号十进制整数、浮点数、科学计数法; 负数通过一元负号处理
布尔常量	true、False	大小写无关, 词法阶段直接归一为布尔值单词
预定义函数	sin、cos、min、max	大小写无关; 函数名本身合法, 但是否跟左括号配对由语法阶段检查
标识符错误	mix、v4、E6	字母开头但不属于布尔常量和预定义函数, 抛 <code>IllegalIdentifierException</code>
非法小数	.11、4.、7e、4.e6	一律在扫描数字时抛 <code>IllegalDecimalException</code>
其他非法符号	@、\$	抛 <code>IllegalSymbolException</code>

3.2 数值扫描自动机

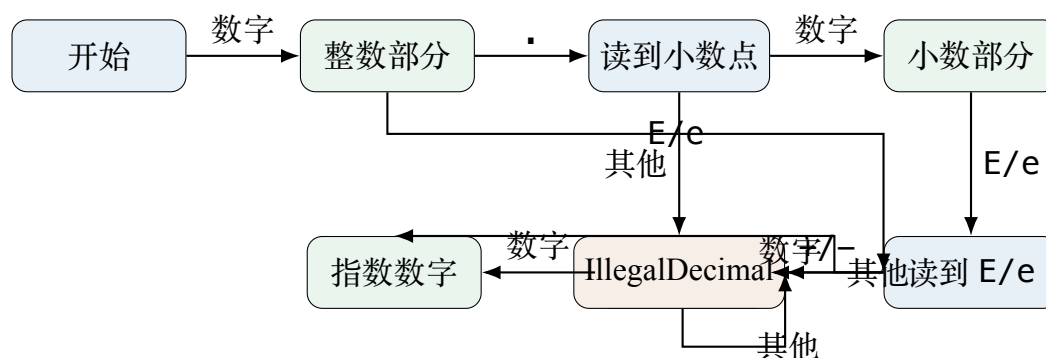


图 1: 数值常量 DFA。起点不是数字时不会进入该自动机, 因此 .11、E6 这类输入会分别落到不同异常。

3.3 词法实现细节

这部分有两个细点必须单列说明:

1. **sin5** 不能在词法阶段误判为非法标识符。实现里先连续读取字母, 再判断这段字母是否是预定义函数名; 如果是, 就立刻返回函数名单词, 把后面的数字留给下一轮扫描。因此 sin5 会被拆成 sin 和 5, 随后在语法阶段因为缺少左括号抛 `FunctionCallException`。

2. 负数不是词法单元。例如 `-3.14` 被拆成 `-` 和 `3.14` 两个单词，是否为合法表达式由语法阶段决定。

Listing 1: Scanner 中的数字与标识符处理逻辑（节选）

```
if (Character.isWhitespace(ch) && ch != ' ') {
    throw new IllegalArgumentException(...);
}
if (ch == '.') {
    throw new IllegalDecimalException(...);
}
if (Character.isDigit(ch)) {
    return readNumber();
}
if (Character.isLetter(ch)) {
    return readIdentifier();
}
```

4 算符优先语法分析与语义处理

4.1 模块划分

模块	职责
Calculator	保留软装置入口，只负责创建扫描器、解析器并返回最终结果
Token	封装词法单元，避免解析器直接看字符
TokenType	负责词法扫描和词法异常分类
Scanner	维护优先级、结合性和归约判定逻辑
OperatorTable	维护运算符栈、值栈、括号/函数帧，执行移进与归约
Parser	封装数值型/布尔型结果，统一做类型检查
Value	

4.2 优先级与结合性

运算符类别	代表符号	结合性	说明
三元条件	?:	右结合	通过 QUESTION 和 TERNARY 两个栈标记配合处理
逻辑或		左结合	不做短路，两个操作数都已在归约前求值
逻辑与	&	左结合	与 PDF 一致，非短路
关系运算	< <= > >= = <>	左结合	输入必须是数值型，结果是布尔型
加减	+ -	左结合	二元运算
乘除	* /	左结合	除法会在归约时检查除数是否为 0
一元运算	- !	右结合	由当前是否“期待操作数”决定 - 是前缀还是中缀
幂运算	^	右结合	处理 2^3^2 这一类右结合场景

4.3 压缩后的 OPP 关系表

栈顶符号类	当前输入符号类	关系	用途
左括号 (	任意普通运算符 / 操作数	<	括号内继续移进
左括号 (	右括号)	=	直接配对并退出当前括号层
问号 ?	分支内部的普通符号	<	继续移进真分支或假分支
问号 ?	冒号 :	=	把 ? 变成待归约的三元标记
高优先级运算符	低优先级运算符	>	先归约栈顶，再处理当前输入
低优先级运算符	高优先级运算符	<	当前输入移进
同优先级左结合运算符	同优先级输入	>	先归约旧运算符
同优先级右结合运算符	同优先级输入	<	保证右结合，如 ^ 和三元条件
栈底 #	输入结束 #	=	分析完成

上表不是把每个终结符都展开成巨大的矩阵，而是按“栈顶类”和“输入类”压缩。代码里对应的判定只有一处：

Listing 2: OperatorTable 中的归约条件（节选）

```
if (stackTop == SENTINEL || stackTop == LEFT_PAREN || stackTop == QUESTION)
{
    return false;
}
if (stackTop.precedence > incoming.precedence) {
    return true;
}
return stackTop.precedence == incoming.precedence
    && incoming.associativity == LEFT;
```

4.4 移进/归约流程

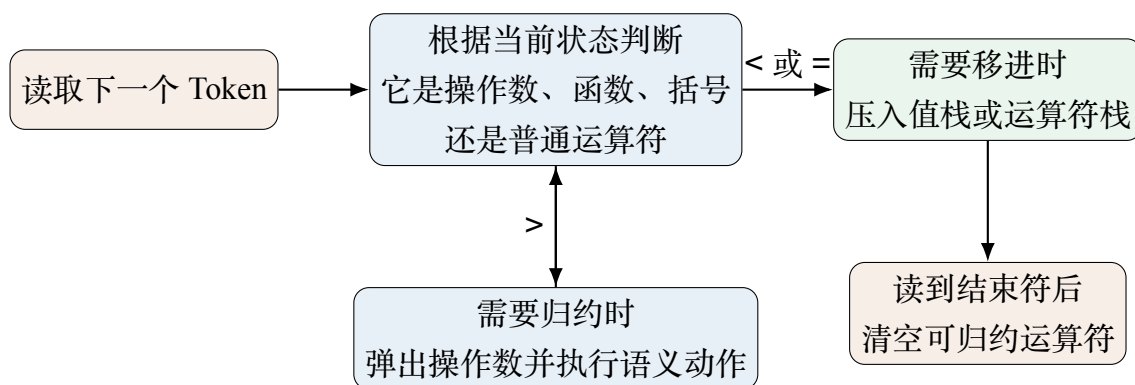


图 2: OPP 控制流程。函数参数和三元配对都放在同一套栈机制中处理。

Listing 3: Parser 主流程伪码

```
operatorStack.push(#)
expectingOperand = true
while token != END:
    if token is operand:
        shift value
    else if token is function name:
        require next token is '('
        push function frame
    else if token is ',' or ')':
        reduce until '('
        collect function argument or close group
    else if token is ':':
        locate nearest unmatched '?'
```

```

        replace it with TERNARY marker
    else:
        while shouldReduce(stackTop, token):
            reduceTop()
        shift operator
    final reduce until '#'

```

4.5 语义规则与异常落点

运算类型	类型规则	归约时的检查
算术运算	number × number → number	任一操作数是布尔型，抛 TypeMismatchedException
关系运算	number × number → boolean	比较前强制取数值
逻辑运算	boolean × boolean → boolean	不短路；两个操作数都会先入栈、再归约
一元负号	number → number	由“当前是否期待操作数”决定其为前缀形式
逻辑非	boolean → boolean	对数值型使用！直接报类型错误
三元运算	boolean, number, number → number	条件必须是布尔型，两个分支必须是数值型
预定义函数	sin/cos: 1 个数值参 数; min/max: 至少 2 个数值参数	参数个数不足抛 MissingOperandException; 参数过多或缺左括号抛 FunctionCallException
最终结果	number	最终栈顶是布尔型时抛 TypeMismatchedException

异常划分也按实验要求收紧：

- 词法阶段只抛 LexicalException 及其派生类；
- 语法阶段只抛 SyntacticException 及其派生类；
- 语义阶段只抛 SemanticException 及其派生类。

- `12.3Emax(4,5,6)` 在扫描指数部分时已经发现错误，因此直接落到 `IllegalDecimalException`。
- `sin5` 会先被切成 `sin` 和 `5` 两个单词，再因为函数名后面没有左括号而抛 `FunctionCallException`。
- `false & (1 / 0 > 0)` 会触发 `DividedByZeroException`，因为逻辑运算不做短路。

5 自定义测试用例说明

`mytest.xml` 共加入 15 个用例，补的是 PDF 里标准测试没有完全覆盖的边界：

- 一元负号与二元减号的组合：`2 - 3 * -4`、`5--2.5`；
- 右结合乘方：`2 ^ 3 ^ 2`；
- 空表达式与非法空白：仅空格输入、带 Tab 的输入；
- 非法数字：`.11 + 1`、`7e + 1`；
- 标识符与函数调用边界：`v4 + 1`、`sin5`、`max5, 6, 8)`；
- 三元配对错误：`false ? 9 : true ? 1 : 3 : 5`；
- 三元类型错误：`12 ? 34 : 56`、`true ? 42.5 > 5 * 8 : 15`；
- 非短路逻辑：`false & (1 / 0 > 0)`；
- 缺失运算符：`cos(0.5)12.3E+4`。

6 程序运行截图

6.1 GUI 运行界面



图 3: 软装置界面渲染图。左图展示正常求值，右图展示非法标识符输入的异常结果。

6.2 测试结果截图

```

standard.xml test result

Testing Procedure
=====

C001 A simple expression.
Input: 9 - 3 * 2
Expected output: 3
Passed !

C002 Expression with arithmetic operations.
Input: 2.25E+2 - (55.5 + 4 * (10 / 2) ^ 2)
Expected output: 69.5
Passed !

C003 Expression with arithmetic operations.
Input: 65 / 5 - 130e-1
Expected output: 0.0
Passed !

C004 Expression with relational and logical operations.
Input: (5 > 3) & (4 < 8) ? 15 : 16
...
...

-----
Statistics Report (16 test cases):

[Passed case(s): 16 (100.0%)
[Warning case(s): 0 (0.0%)
[Failed case(s): 0 (0.0%)
=====

```

图 4: standard.xml 测试结果摘要，16 个用例全部通过。

```

simple.xml test result

Testing Procedure
=====

C001 A simple expression.
Input: 9 - 3 * 2
Expected output: 3
Passed !

C002 Expression with arithmetic operations.
Input: 2.25E+2 - (55.5 + 4 * (10 / 2) ^ 2)
Expected output: 69.5
Passed !

C003 Expression with arithmetic operations.
Input: 65 / 5 - 130e-1
Expected output: 0.0
Passed !

E001 Left parenthesis expected.
Input: (2 + 3) ^ 3) - ((1 + 1)
...
...

-----
Statistics Report (8 test cases):

[Passed case(s): 8 (100.0%)
[Warning case(s): 0 (0.0%)
[Failed case(s): 0 (0.0%)
=====

```

```

mytest.xml test result

Testing Procedure
=====

C101 Unary minus mixed with multiplication.
Input: 2 - 3 * -4
Expected output: 14
Passed !

C102 Repeated minus should be parsed as binary minus plus unary minus.
Input: 5-2.5
Expected output: 7.5
Passed !

C103 Power is right associative.
Input: 2 ^ 3 ^ 2
Expected output: 512
Passed !

E101 Spaces only should be treated as an empty expression.
Input:
...
...

-----
Statistics Report (15 test cases):

[Passed case(s): 15 (100.0%)
[Warning case(s): 0 (0.0%)
[Failed case(s): 0 (0.0%)
=====

```

图 5: simple.xml 与 mytest.xml 测试结果摘要，分别通过 8/8 和 15/15。

7 文档与源程序组织说明

路径 / 文件	内容
src/parser/	
Calculator.java	软装置入口, 保持 <code>calculate</code> 接口不变
src/parser/	
Scanner.java	词法扫描与词法异常分类
src/parser/	
Parser.java	OPP 主控制程序、函数参数帧和归约动作
src/parser/	
OperatorTable.java	优先级和结合性表
src/parser/	
Token.java	
TokenType.java	单词模型
src/parser/	
Value.java	数值型 / 布尔型封装
testcases/	
simple.xml	
standard.xml	
mytest.xml	三套测试用例
doc/	Javadoc 生成结果
readme.txt	提交说明和命令入口
design.pdf	按实验要求命名的正式实验报告

8 实验心得

这次重做里, 真正花时间的不是算术语义, 而是“异常必须落到最具体的类”这一点。几个具体体会如下:

1. 词法阶段和语法阶段的职责必须切干净。像 `12.3Emax(4,5,6)` 这种输入, 如果数字扫描写得松, 后面就只能在语法阶段补锅, 异常类型也会错。
2. 三元运算不能只靠优先级解决, 还要显式处理 `?` 和 `:` 的配对。额外的 `:` 如果先触发语义归约, 就会把本该是 `TrinaryOperationException` 的输入误报成类型错误。
3. OPP 在这个实验里比递归下降更合适。PDF 本身要求展示算符优先表, 函数参数、右结合幂运算和三元条件也都能放进同一套栈框架里处理, 报告和实现是一致的。

`simple.xml` 通过 8/8, `standard.xml` 通过 16/16, `mytest.xml` 通过 15/15。Javadoc、`readme.txt`、`design.pdf` 和报告图示已同步补齐。